TripCraft — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and who writes what

Five working nodes plus a supervisor, around one shared state. Decide **who writes each field** before you write any node — that's the whole design:

State field	Only writer	What it holds
`notes`	research	attractions, each with a `[source]` tag
`day_plan`	analysis	2–4 themed days
`draft`	writer	the itinerary
`score`, `feedback`	reviewer	the checked verdict
`routing_log`	supervisor	one `{step, reason}` per decision

Files: `state.py` (TeamState TypedDict), `schemas.py` (ReviewVerdict), `agents/` (one file per specialist + `supervisor.py`), `graph.py` (wires them), `tools/` → research reads `data/attractions.json`.

Hint 2 — the research tool and the unknown-destination stop

Research must read a file, not the model's memory:

```
@tool
def lookup_attractions(destination: str) -> list:
    """Look up real attractions for a destination from the local file."""
    with open(ATTRACTIONS_PATH) as f:
        data = json.loads(f.read())
    return data.get(destination.strip().lower(), [])
```

An unknown destination returns `[]`. That empty list is the signal the supervisor uses to stop the whole run cleanly — so handle "notes is `[]`" as a distinct case, separate from "notes is `None`" (not looked up yet).

Hint 3 — the supervisor routes by reading the state

Don't ask the model who goes next — decide it in code from which fields are filled. A clean `decide_next(state)` ladder:

```
notes is None            -> research
notes == []              -> finish (unknown destination)
day_plan is None         -> analysis
draft is None            -> writer
score is None            -> reviewer
score >= PASS_SCORE      -> finish (accepted)
revisions >= MAX_REVISIONS -> finish (couldn't agree)
otherwise                -> revise (back to writer)
```

Each specialist node returns `Command(goto="supervisor", update={...})`, so control always comes back to the supervisor. The supervisor returns `Command(goto=<specialist>)` or `Command(goto=END)`. The graph has exactly one edge: `START -> supervisor`.

The revision subtlety: when the supervisor sends a rejected draft back to the writer, it must **clear `score` back to `None`** and bump `revisions` in the same `Command` update — otherwise the reviewer never runs again (its score is still set) and the loop can't terminate correctly.

Hint 4 — the checked verdict, and keeping the loop honest

Make the verdict a shape the code can trust:

```
class ReviewVerdict(BaseModel):
    score: int = Field(ge=1, le=10)
    feedback: str
```

Build the reviewer as `prompt | model.with_structured_output(ReviewVerdict)`. Keep `PASS_SCORE` and `MAX_REVISIONS` **in code**, never in the prompt — they're your decisions, and the model shouldn't be able to drift on them. Test the supervisor on hand-made states (no model needed): fresh → research, empty notes → stop, score 9 → accept, score 4 at the cap → give up. That one unit test proves the whole control flow before you ever spend a token.

Still stuck? The whole thing, every file, is in the complete solution.